

Stock market forecasting using DRAGAN and feature matching

Fateme Shahabi Nejad, Mohammad Mehdi Ebadzadeh *

Department of Computer Engineering, Amirkabir University of Technology (Tehran Polytechnic), Tehran, Iran

ARTICLE INFO

Keywords:

Stock price prediction
Time series forecasting
Generative Adversarial Networks
Feature matching

ABSTRACT

Applying machine learning methods to forecast stock prices has been a topic of interest in recent years. However, a few studies have been reported based on generative adversarial networks (GANs) in this area, but their results are promising. While GANs are powerful generative models successfully applied in different areas, they suffer from inherent challenges such as training instability and mode collapse. Another primary concern is capturing correlations in stock prices. Therefore, the main challenges fall into two categories: capturing correlations and addressing the inherent problems of GANs. In this paper, we introduce a novel framework based on DRAGAN¹ and feature matching for stock price forecasting, which improves training stability and alleviates mode collapse. We employ windowing to acquire temporal correlations by the generator and exploit conditioning on discriminator inputs to capture temporal correlations and correlations between prices and features. Experimental results on data from several stocks indicate that proposed method outperforms long short-term memory (LSTM) as a baseline method, as well as basic GANs and WGAN-GP² as two different variants of GANs.

1. Introduction

Stock price forecasting is challenging as it is influenced by various factors such as economic components, political issues, and the company's financial performance. Traditional methods of stock price forecasting can be broadly categorized into two categories: technical analysis and fundamental analysis. The technical analysis predicts future stock prices using historical data such as stock price and trading volume and captures the future trends of the stock market. Among the prevalent methods of technical analysis, we can mention moving average (MA), technical indicators, support and resistance levels, trend lines, candlestick patterns, and channels. Fundamental analysis typically deals with qualitative data like financial news and economic data. It evaluates the company's stock based on its intrinsic value, financial reports, and market situation. Fundamental analysis is a reasonable methodology for long-term investment and is generally unaffected by short-term news. In contrast, technical analysis is more suitable for short-term trading and is usually influenced by news.

Forecasting stock prices using machine learning methods has become an area of interest in recent years. These methods can be categorized into classic, traditional machine learning, and deep learning methods, with deep learning being the most popular. Among deep learning

methods, GANs have shown immense potential in learning the distribution of high-dimensional and complex data. Using an adversarial approach, GANs operate without specific assumptions about the underlying distribution and can generate realistic samples from a latent space. This remarkable characteristic has led to their application in various domains. However, GANs have received less attention in stock forecasting than other deep learning techniques. This can be attributed to their challenging and heuristic training process, which often encounters instability and mode collapse. Additionally, most existing studies based on GANs rely on a small amount of data and simplistic structures, limiting their competitiveness against other hybrid deep learning approaches. In the context of stock prediction, we face not only the inherent challenges of GANs, such as mode collapse and training instability, but also the task of effectively capturing the dependencies within the time series data.

We propose an integrated framework that alleviates each challenge by introducing novel approaches. These include customized cost functions based on DRAGAN, feature matching, conditioning, and windowing techniques. Our proposed framework outperforms previous studies based on WGAN-GP, basic GANs, and LSTM models. However, it should be noted that further improvements are needed to enhance the results compared to more powerful hybrid deep learning methods.

* Corresponding author.

E-mail addresses: fa.shahabi@aut.ac.ir (F. Shahabi Nejad), ebadzadeh@aut.ac.ir (M.M. Ebadzadeh).

¹ Deep Regret Analytic Generative Adversarial Network.

² Wasserstein GAN – Gradient Penalty.

Nevertheless, the promising outcomes of this study can serve as motivation for future advancements in GAN-based research. This paper presents a novel forecasting framework based on DRAGAN, leveraging historical data and technical indicators. The significant contributions of this work are as follows:

- To our knowledge, this paper is the first to employ DRAGAN and feature matching for forecasting stock prices.
- Feature matching and customized cost functions are applied to design the proposed framework, which mitigates mode collapse³ and improves training stability.
- Utilizing windowing and conditioning can better capture temporal correlations and correlations between prices and features.
- Proposed method is compared to LSTM as a baseline method, as well as basic GANs and WGAN-GP as two different variants of GANs. Experiments show that it outperforms the compared methods. Indeed, the proposed approach can better learn the future trends of stock.
- Since the proposed framework is designed to capture correlations, it may successfully use for similar tasks in the time series area.

The organization of this paper is as follows: [Section 2](#) provides an overview of the research conducted for stock price forecasting based on the machine learning attitude. [Section 3](#) discusses the theoretical concepts of the paper briefly. [Section 4](#) first introduces the overall structure of the proposed framework and then examines its components in more detail. The experimental results of the proposed method and its comparison with baseline methods are presented in [section 5](#). Finally, concluding remarks and suggestions for future research are given in [Section 6](#).

2. Related works

Recently, there has been an increasing trend in using machine learning methods to forecast stock prices. Various techniques have been employed for stock forecasting, ranging from regression methods to artificial neural networks (ANNs). Most of these methods rely on machine learning and technical analysis, with fundamental analysis rarely used. The reason is that technical data is available daily and even hourly, while fundamental data is reported quarterly and annually. Since the nature of technical data is quantitative and structured, some classic time series algorithms have been extended to forecast the stock price by considering it as a time series. After reviewing conducted studies in this area, we have summarized them into the following three categories: classic methods, traditional machine learning methods, and deep learning methods.

Classic methods: By considering the stock prices as time series, some statistical time-series methodologies such as auto-regression (AR), auto-regressive moving average (ARMA), and auto-regressive integrated moving average (ARIMA) have been developed to forecast the stock price ([Box et al., 2015](#)). However, ARIMA alone does not perform well due to the non-linearity of stock data. [Areekul et al. \(2009\)](#) employed a neural network to improve the performance of ARIMA. [Pai and Lin \(2005\)](#) suggested the combination of ARIMA and support vector machine (SVM) for stock price forecasting. Generally, classic methods have inferior results compared to the other two categories.

Traditional machine learning methods: Traditional approaches include methods such as naive Bayes, SVM, random forest, k-nearest neighbor (k-NN), fuzzy theory, and linear regression. [Fenghua et al. \(2014\)](#) decomposed the stock price into features using singular spectrum analysis (SSA). Then an SVM applied these features for price prediction.

[Patel et al. \(2015\)](#) evaluated the performance of four machine learning methods, including ANN, SVM, random forest, and naive Bayes, for forecasting the Indian stock market. [Bhuriya et al. \(2017\)](#) applied linear regression to predict the stock market and found that it outperformed the polynomial and radial basis function (RBF) regression models. [Ebadati and Mortazavi \(2018\)](#) suggested a combined method based on genetic algorithms and ANNs to predict stock prices. [Sinaga et al. \(2019\)](#) proposed an effective mixture model for stock trend prediction. They modified support vector k-nearest neighbor clustering (SV-kNNC) by replacing the k-means clustering with a self-organizing map (SOM). [Zhang et al. \(2019\)](#) presented a new prediction method using fuzzy time series and genetic algorithms. [Sedighi et al. \(2019\)](#) introduced a novel prediction model that uses an adaptive neuro-fuzzy inference system (ANFIS), an artificial bee colony algorithm, and an SVM. [Wang et al. \(2019\)](#) extended a new hybrid model based on fractal interpolation and SVM models to forecast time series of stock price indexes. [Basak et al. \(2019\)](#) predicted stock price direction using random forest and gradient boosted decision trees (using XGBoost). [Cao et al. \(2019\)](#) utilized a complex network method and then used network characteristic variables as input variables for k-NN and SVM to predict stock price patterns. [Nasiri and Ebadzadeh \(2022\)](#) proposed a multi-functional recurrent fuzzy neural network (MFRFNN) based on two neural networks for chaotic time series prediction. They compared the proposed model with the decision tree, random forest, support vector regression (SVR), multi-layer perceptron (MLP), and LSTM.

Deep learning methods: With the development of deep learning, there has been increased attention to using this approach for stock price forecasting. Deep learning can learn complex non-linear structures and suitable features without experts' prior knowledge ([Chong et al., 2017](#)). [Singh and Srivastava \(2017\)](#) employed principal component analysis (PCA) with a deep neural network (DNN) and demonstrated that deep learning could improve the accuracy of stock price forecasting.

The Convolutional neural network (CNN) is one of the most successful types of networks in deep learning, which has extensive applications in image processing, natural language processing, speech recognition, and more. [Cao and Wang \(2019\)](#) used a CNN and a CNN + SVM to predict stock index values. Empirical results indicated that both prediction models were efficient.

The LSTM network has shown promising results for time series forecasting, and many studies have been conducted based on LSTM or combined models incorporating it. [Di Persio and Honchar \(2017\)](#) compared the performance of different recurrent neural networks (RNNs), including the basic multi-layer RNN, LSTM, and gated recurrent unit (GRU). Their results for predicting Google's stock price showed that LSTM performed better than other variants. [Fischer and Krauss \(2018\)](#) applied LSTM networks to a large-scale market prediction task on the S&P 500 index. They demonstrated that it outperformed random forest, DNN, and logistic regression. [Chen and Ge \(2019\)](#) found that using the attention mechanism could improve the performance of the LSTM-based method. [Jin et al. \(2020\)](#) suggested that incorporating investors' sentiment in stock prediction can improve performance. They also applied an attention mechanism in combination with LSTM to extract meaningful information. [Xu et al. \(2020\)](#) employed the wavelet transform to reduce noise in stock data and a stacked auto-encoder to remove unessential features. Finally, they applied a bidirectional LSTM to predict the stock price. [Gunduz \(2021\)](#) combined LSTM with a variational auto-encoder (VAE) to predict the price of banking stocks in Borsa Istanbul.

The proper performance of deep networks usually depends on the amount of available training data. Therefore, several studies have utilized transfer learning to overcome problems such as insufficient training samples and overfitting. [Nguyen and Yoon \(2019\)](#) leveraged the advantages of transfer learning in combination with LSTM to mitigate the challenges mentioned earlier. [Gu and Dai \(2021\)](#) presented a new multi-source transfer learning algorithm to address the problems of insufficient training data and differences in distribution between old and new data.

³ The mode collapse problem occurs when the generator maps different inputs to the same output. In other words, the generator produces only a particular sample or a limited variety of samples.

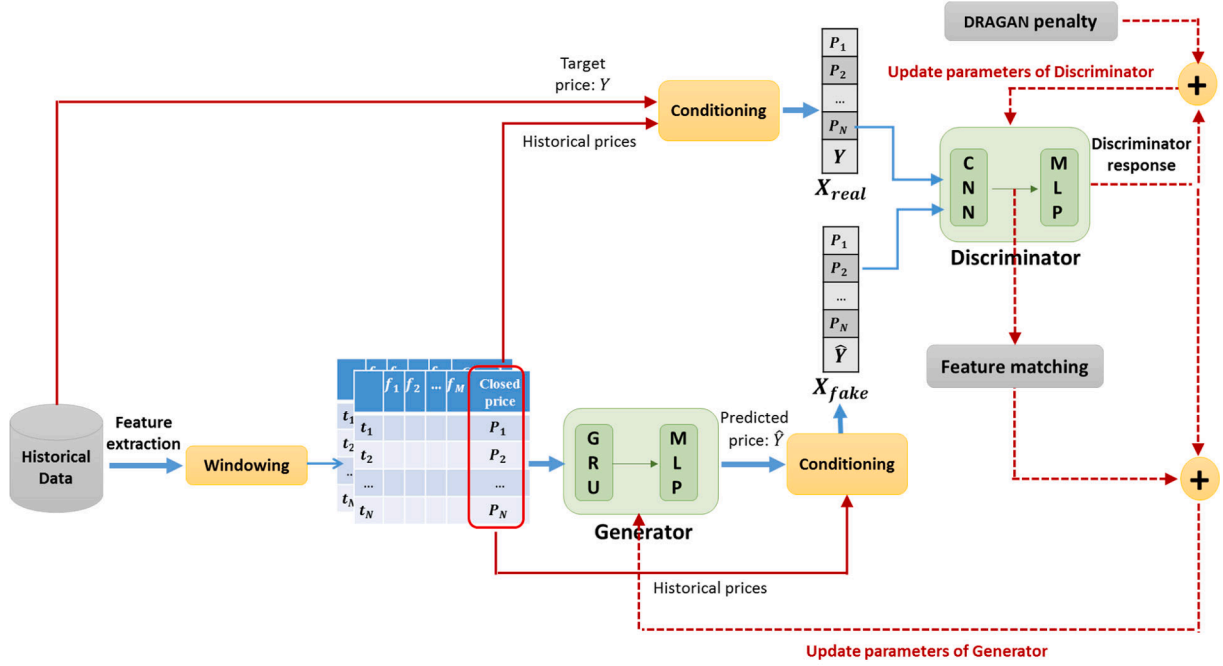


Fig. 1. The overall structure of proposed framework.

Several studies have attempted to adapt GANs for stock price prediction. Zhou et al. (2018) proposed basic GANs with LSTM as a generator and CNN as a discriminator for adversarial forecasting of stock prices. They modified the loss function of basic GANs by adding direction prediction loss and forecast error loss. Their experiments showed that GANs outperformed other models such as ARIMA, ANN, SVM, and LSTM. Zhang et al. (2019) suggested basic GANs with LSTM as the generator and MLP as the discriminator to predict closing prices. They demonstrated that GANs had better results than SVR, ANN, and LSTM. Sonkiya et al. (2021) made a sentiment analysis of news related to Apple Inc. Then, they used basic GANs to predict the stock price based on sentiment scores and other relevant features. The proposed method performed better than benchmark models such as ARIMA, GRU, LSTM, and basic GANs (without sentiment analysis). Lin et al. (2021) used news sentiment analysis and WGAN-GP to predict stock prices. They employed GRU for the generator and CNN for the discriminator. Examining the results on Apple stock showed that WGAN-GP outperformed LSTM, GRU, and basic GANs in terms of performance.

A review of studies demonstrates that deep learning methods have been more successful than traditional and classic methods (Bustos & Pomares-Quimbaya, 2020; Kumbure et al., 2022). Among them, LSTM is the most widely used method for stock price forecasting. Since the employment of GANs for stock prediction faces challenges, relatively few studies have been reported based on them. However, the results of these studies are promising. Therefore, we propose a novel GANs-based method and compare its results with successful models such as LSTM, WGAN-GP, and Basic GANs.

3. Backgrounds

3.1. GANs

After the introduction of GANs in 2014 (Goodfellow et al., 2014), their employment in various fields of machine learning has rapidly expanded. GANs consist of two models, the generator and the discriminator. Generator G aims to produce real-like samples that the discriminator cannot recognize as fake, while discriminator D intends to distinguish real samples from fake ones. In an adversarial learning process, the two models reach an equilibrium point where the generator

can capture real data distribution, and the discriminator cannot distinguish between fake and real samples. The training process includes simultaneous optimization of D and G in a two-player game. The value function for the basic GAN is defined as follows:

$$\min_G \max_D V(D, G) = E_{x \sim P_r} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))] \quad (1)$$

Here, x represents samples of real data under the P_r distribution. The generator inputs a random noise vector, z , $p_z(z)$, and produces fake samples, $G(z)$. $D(\cdot)$ is the output of the discriminator, which expresses the probability of the input being real as a single scalar.

3.2. Long Short-Term memory (LSTM)

LSTM models are a particular type of RNN that can learn long-term dependencies and have been successful in time series processing. Hochreiter and Schmidhuber (1997) introduced LSTM to address the vanishing and exploding gradients associated with RNN. An LSTM cell comprises three components: the forget gate, input gate, and output gate. The forget gate decides whether information should be retained or discarded, while the input gate controls what new information can be added to the cell. Finally, the output gate determines what data should be sent to the next cell.

3.3. Gated recurrent units (GRUs)

GRUs are a simplified version of LSTM introduced by Cho et al. (2014). Since GRUs include only the reset gate and the update gate, they are faster than LSTMs. The reset gate can capture short-term dependencies and determines how much previous information should be forgotten. The update gate can capture long-term dependencies and has the same functionality as the forget gate and input gate of an LSTM. It decides what information to discard and what new information to add. We cannot express an unequivocal opinion on the preference of LSTM or GRUs. Researchers typically investigate both models and choose the appropriate one based on their application.

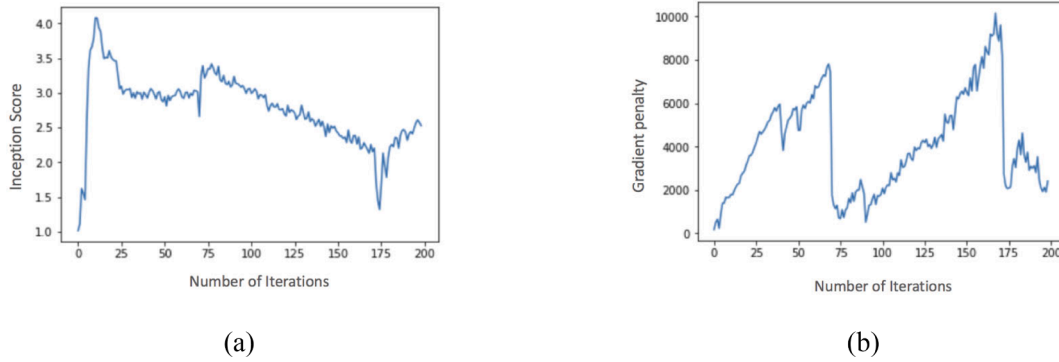


Fig. 2. One hidden layer network to model MNIST. (a) Inception score against time during training of basic GANs. (b) The squared norm of discriminator's gradients around real data points for the same experiment. (Kodali et al., 2017).

4. Proposed method

This section presents the proposed method for forecasting the closed price of a specific stock based on its historical data and several technical indicators. Our initial investigations revealed that applying naive GANs to time series data yields unsatisfactory results. Specifically, naive GANs struggle to capture temporal correlations and correlations between price and features. Additionally, GANs are susceptible to inherent challenges such as training instability and mode collapse. Therefore, the main challenges fall into two categories: capturing correlations and addressing the inherent problems of GANs. In this paper, a novel framework has been suggested to mitigate these challenges. The overall structure of this framework is shown in Fig. 1.

The historical data includes stock information such as High, Low, Open, Close, Adjacent close, and Volume. First, technical indicators are extracted from the historical data. The combination of historical data and technical indicators makes the features denoted by $f_1, f_2, \dots, f_M$ in Fig. 1. Next, the acquired data is segmented based on the windowing process and fed to the generator as input. The generator produces the predicted price, denoted by $\hat{Y}$, based on consecutive segments.

In the conditioning step, historical prices and predicted price are concatenated, and the result is fed to the discriminator as input X_{fake} . Historical prices represent the closed prices for N consecutive time steps in a segment, where N is the size of the segment (or the window length). Similarly, the concatenation of historical prices and the target price is fed to the discriminator as input X_{real} . The generator and discriminator are adversarially trained based on our customized cost functions. In the following sections, we will describe the details of the proposed framework.

4.1. Windowing

The windowing technique is applied to capture temporal correlations and correlations between prices and features. It involves rolling a window over both the training and test sets and extracting time series segments with a specific length. Therefore, each segment comprises historical prices and features for N consecutive time steps. This way, we feed these segments as inputs to the generator instead of a single time step. By doing so, the generator can learn many-to-one or many-to-many mappings instead of just one-to-one mappings. It can predict future prices based on segments of historical prices and features. For instance, the model can input three-day time series and forecast prices for the next two days.

4.2. Conditioning

The second approach for capturing correlations involves using conditional GANs. Mirza and Osindero (2014) introduced a conditional variant of GANs in which the generator and discriminator are

conditioned on some auxiliary information. This information can be any type of data, such as class labels or other auxiliary data. Conditioning can be achieved by feeding this extra information into both the discriminator and the generator.

In this paper, conditioning is performed by concatenating historical prices with the predicted price and feeding the resulting as input to the discriminator. Another discriminator input is provided by concatenating historical prices with the target price. Here, historical prices act as auxiliary information. Using conditioning can assist in capturing temporal correlations and correlations between prices and features, leading to improved accuracy in forecasting prices.

4.3. Applying DRAGAN

Although GANs are powerful generative models, they are susceptible to training instability and mode collapse. Numerous advancements have been made over the years to mitigate these challenges. This study explores how to leverage state-of-the-art GANs to facilitate stock forecasting. WGAN⁴ is one of these popular models introduced by Arjovsky et al. (2017). They replaced the Jensen-Shannon divergence used in the basic GANs with the Earth-Mover distance (also known as the Wasserstein distance). The Earth-Mover distance is the minimum cost of transporting mass in order to transform the source distribution into the destination distribution (Arjovsky et al., 2017). The cost is defined as the mass multiplied by the transport distance. The WGAN cost function is simplified using the Kantorovich-Rubinstein duality (Villani, 2009) as follows:

$$\min_G \max_{D \in \mathcal{D}} \mathbb{E}_{X_{real} \sim P_r} [D(X_{real})] - \mathbb{E}_{X_{fake} \sim P_g} [D(X_{fake})] \quad (2)$$

where P_g and P_r are the generated and real data distributions, respectively. In WGAN, the discriminator is called a critic because it outputs a scalar score instead of a probability. This score, denoted by $D(X)$, represents how real the input is. Here, $\mathcal{D}$ is the set of 1-Lipschitz functions. A function is considered 1-Lipschitz if and only if its gradients have a norm of at most one everywhere (Gulrajani et al., 2017). To enforce the 1-Lipschitz constraint on the critic, WGAN clips the weights to fall within the specified range $[-c, c]$. The weight clipping can cause optimization problems and non-convergence. Therefore, WGAN-GP proposes another method to enforce the 1-Lipschitz constraint. It penalizes the critic if the gradient norm deviates from the value of 1 (Gulrajani et al., 2017). The WGAN-GP cost function is specified by:

$$\min_D \mathbb{E}_{X_{fake} \sim P_g} [D(X_{fake})] - \mathbb{E}_{X_{real} \sim P_r} [D(X_{real})] + \lambda \cdot \mathbb{E}_{\tilde{X} \sim P_{\tilde{X}}} [(\|\nabla_{\tilde{X}} D(\tilde{X})\|_2 - 1)^2] \quad (3)$$

⁴ Wasserstein GAN.

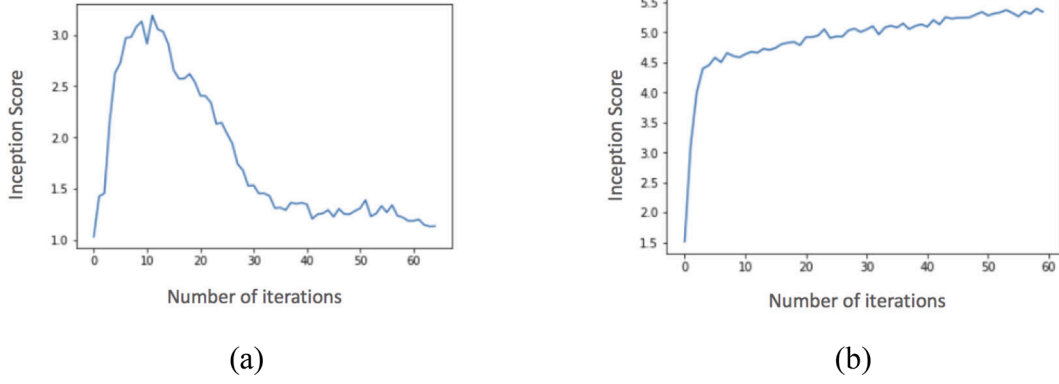


Fig. 3. One hidden layer network to model MNIST. (a) Inception score plot for basic GANs. (b) Inception score plot after applying DRAGAN regularization term. (Kodali et al., 2017).

$$\hat{X} \leftarrow \epsilon X_{real} + (1 - \epsilon) X_{fake}, \epsilon \sim U[0, 1]$$

$\hat{X}$ is any point interpolated based on samples from P_g and P_r , where ϵ is a random number selected from the uniform distribution $U[0, 1]$. The λ coefficient specifies the weight of the gradient penalty term. Experiments demonstrate that WGAN-GP enhances training stability and performs better than basic GANs and WGAN.

One of the major concerns in the training process of GANs is the issue of mode collapse. DRAGAN proposed a new perspective to address mode collapse. It is one of the well-known cost functions for training GANs, as proposed by Kodali et al. (2017). This model assumed that mode collapse occurs due to the convergence of the GAN game to undesirable local equilibria. After studying empirical cases of mode collapse, the paper's authors found that mode collapse situations are often accompanied by sharp gradients of the discriminator function around the real points (Fig. 2). This figure shows the changes in the discriminator's gradients during mode collapse occurrences. Indeed, mode collapse points in the inception score plot are associated with sudden changes in the discriminator's gradients. Such sharp gradients encourage the generator to map multiple input vectors to the same output (defining mode collapse), compelling the game toward an undesirable equilibrium.

Therefore, a simple strategy to alleviate mode collapse is to constrain the gradients of the discriminator. This way, the discriminator is encouraged to escape from local minima. Hence, they suggested a novel gradient penalty that penalizes the discriminator gradients near the real data points:

$$\lambda E_{X_{real} \sim P_r, \delta \sim N_d(0, cI)} [\|\nabla_x D_\theta(X_{real} + \delta)\|_2 - k]^2 \quad (4)$$

Here, $X_{real} + \delta$ represents the noisy real point sample, where δ is a random number chosen from the Gaussian distribution $N_d(0, cI)$. The hyperparameter k specifies the permitted range for the gradients norm. Fig. 3 shows the results of employing the DRAGAN penalty to a toy example with a single hidden layer. The inception score plot in (b) indicates that adding the DRAGAN regularization term mitigates mode collapse.

In comparison to WGAN-GP and basic GANs, this strategy improves training stability. Their experimental results demonstrated that DRAGAN enabled faster training and revealed less mode collapse. Furthermore, it led to better stability across various architectures and objective functions. From another perspective, penalizing samples in the local region encourages the discriminator to behave linearly around the local data manifold. The convexity property of linear functions assists the optimization process throughout training to overcome the gradient exploding problem and non-convergence.

The essential difference between WGAN-GP and DRAGAN is the region where the gradient penalty term is applied. When the generator produces unsatisfactory results (i.e., the generated samples are distant

from the real samples), the WGAN-GP's gradient penalty has more drawbacks due to poor interpolation. Thus, we employ the DRAGAN idea in developing the framework and integrate its penalty term into the cost function. Experiments demonstrate that this leads to improved stability during training and better performance than WGAN-GP.

4.4. Feature matching

The training of GANs aims to find the Nash equilibrium⁵ of a non-convex game involving two agents. Gradient descent methods are suitable for solving traditional optimization problems based on a single agent, not multiple agents. Therefore, using these methods for training GANs may result in non-convergence (Arjovsky & Bottou, 2017). Unfortunately, no appropriate alternative method exists for optimizing GANs, and researchers have only proposed techniques to alleviate these challenges. To enhance GANs optimization, Salimans et al. (2016) suggested several techniques based on a heuristic understanding of the non-convergence problem. Among them, feature matching is employed to stabilize training. For this purpose, the following expression is added to the generator's cost function:

$$\|E_{X_{real} \sim P_r} f(X_{real}) - E_{X_{fake} \sim P_g} f(X_{fake})\|_2^2 \quad (5)$$

In this equation, $f(x)$ indicates the output of a specific intermediate layer of the discriminator for the input x . Since the discriminator distinguishes real data from generated data, $f(x)$ is a suitable representation of the features.

Feature matching intends to minimize the difference between real and generated data features. Investigations demonstrate that feature matching can alleviate the unstable behavior of GANs and improve forecasting accuracy. It guides the generator to produce data that matches the statistics of the real data by considering not only the last layer output of the discriminator but also the intermediate layers, thereby avoiding overtraining. As depicted in Fig. 1, we simultaneously utilize feature matching and discriminator response to design the generator's cost function.

4.5. Cost functions and networks architecture

The proper design of cost functions, generator architecture, and discriminator architecture are fundamental components in the success of GANs-based models. The generator aims to learn the stock price distribution by taking a window of consecutive data and producing the

⁵ According to game theory, the convergence of GANs occurs at the Nash equilibrium point. The Nash equilibrium is the optimal point for the min-max equation of GANs. That is, no player can decrease their cost function without changing the parameters of other players.

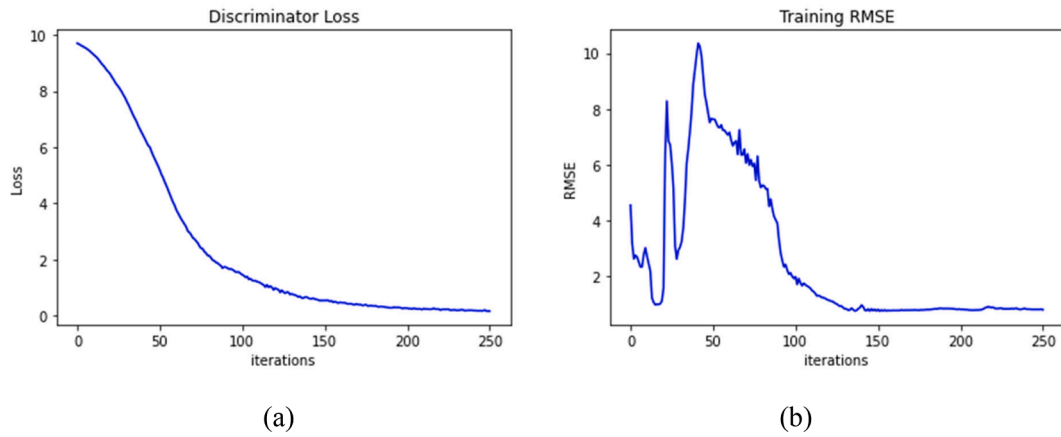


Fig. 4. The training process of the proposed method for Apple stock. (a) The discriminator loss during training. (b) The RMSE error during training.

Table 1

Results of the proposed method compared to the baseline methods (prediction 3 to 1).

Stocks		Proposed method			WGAN-GP			Basic GANs			LSTM		
		RMSE	MAE	R2-score	RMSE	MAE	R2-score	RMSE	MAE	R2-score	RMSE	MAE	R2-score
Apple	Train	0.84	0.67	0.99	0.43	0.32	1.00	0.41	0.30	1.00	0.37	0.26	1.00
	Test	0.92	0.68	0.98	0.95	0.69	0.99	0.98	0.72	0.99	1.00	0.71	0.98
Microsoft	Train	0.83	0.63	0.99	0.98	0.76	0.99	0.75	0.55	0.99	0.83	0.78	0.99
	Test	1.80	1.21	0.98	1.86	1.33	0.98	2.01	1.44	0.99	2.08	1.32	0.98
Amazon	Train	0.69	0.57	1.00	0.55	0.37	1.00	0.76	0.67	0.99	0.54	0.41	1.00
	Test	1.71	1.10	1.00	1.79	1.25	0.99	1.75	1.20	0.99	1.71	1.23	0.98
Nvidia	Train	0.39	0.28	0.99	0.48	0.33	0.99	0.35	0.26	0.99	0.68	0.37	0.99
	Test	1.68	1.21	0.98	1.76	1.29	0.98	1.82	1.35	0.98	1.71	1.25	0.98
Google	Train	0.86	0.68	0.99	0.55	0.40	1.00	0.52	0.38	1.00	0.44	0.31	1.00
	Test	1.09	0.81	0.97	1.21	0.95	0.96	1.26	1.03	0.95	1.19	0.90	0.96
Tesla	Train	0.61	0.53	0.99	0.36	0.25	1.00	0.35	0.25	1.00	0.87	0.69	0.98
	Test	0.80	0.57	0.98	0.84	0.64	0.97	0.87	0.68	0.97	0.85	0.66	0.96
AAL	Train	0.98	0.79	1.00	0.94	0.58	0.99	0.86	0.59	1.00	0.83	0.72	0.99
	Test	1.11	0.84	0.99	1.12	0.79	0.98	1.19	0.5	0.97	1.01	0.85	0.98
Sony	Train	0.86	0.67	0.99	0.59	0.48	0.99	0.58	0.44	0.98	0.64	0.58	1.00
	Test	1.10	0.84	0.98	1.35	1.02	0.98	1.28	0.99	0.97	1.16	0.74	0.98
Intel	Train	0.51	0.39	0.99	0.59	0.46	0.99	0.69	0.58	0.98	0.60	0.50	0.99
	Test	1.02	0.77	0.98	1.07	0.84	0.98	1.11	0.83	0.97	1.19	0.89	0.97
AMD	Train	0.70	0.60	0.96	0.25	0.17	0.99	0.27	0.19	0.99	0.24	0.17	0.99
	Test	1.12	0.74	0.98	1.14	0.76	0.98	1.25	0.85	0.98	1.64	1.21	0.97
Ford	Train	0.28	0.21	0.98	0.29	0.22	0.98	0.30	0.23	0.98	0.23	0.17	0.99
	Test	0.20	0.15	0.98	0.20	0.15	0.98	0.21	0.17	0.97	0.21	0.16	0.97
General Motors	Train	0.62	0.48	0.99	0.64	0.50	0.99	0.65	0.50	0.98	0.57	0.44	0.99
	Test	0.71	0.51	0.95	0.75	0.56	0.94	0.72	0.53	0.95	0.75	0.55	0.94

predicted price. As mentioned earlier, LSTM and GRU can model time series correlations effectively. Preliminary evaluations show that GRU performs better than LSTM in our framework and is also faster. Therefore, two layers of GRU are used for the generator architecture with 256

term and the Wasserstein distance is applied to define the discriminator's cost function. These two terms are added together with the weighting parameter λ_1 as follows:

$$\min_D \mathbb{E}_{X_{fake} \sim P_g} [D_\theta(X_{fake})] - \mathbb{E}_{X_{real} \sim P_r} [D_\theta(X_{real})] + \lambda_1 \cdot \mathbb{E}_{X_{real} \sim P_r, \delta \sim N_d(0, I)} [\|\nabla_X D_\theta(X_{real} + \delta)\|_2 - k]^2 \quad (6)$$

and 128 units, respectively. Next, two fully connected layers are added on top of this architecture to map the data of the GRU layers to the predicted prices. The discriminator intends to distinguish real data from generated data, taking them as inputs and returning a scalar score that determines how real the input is. Due to CNN's success in classification tasks, the discriminator architecture consists of three convolutional layers with 32, 64, and 128 units, respectively, along with two fully connected layers on top of them.

As previously noted, DRGAN enhances training stability and mitigates mode collapse issues. Therefore, combining the DRGAN penalty

The generator's cost function is defined by adding the feature matching expression to the discriminator response:

$$\min_G -\mathbb{E}_{X_{fake} \sim P_g} [D_\theta(X_{fake})] + \lambda_2 \cdot \|E_{X_{real} \sim P_r} f(X_{real}) - E_{X_{fake} \sim P_g} f(X_{fake})\|_2^2 \quad (7)$$

Here, the λ_2 parameter determines the weight of feature matching. In Fig. 1, dashed lines indicate the backpropagation of gradients corresponding to proposed cost functions.

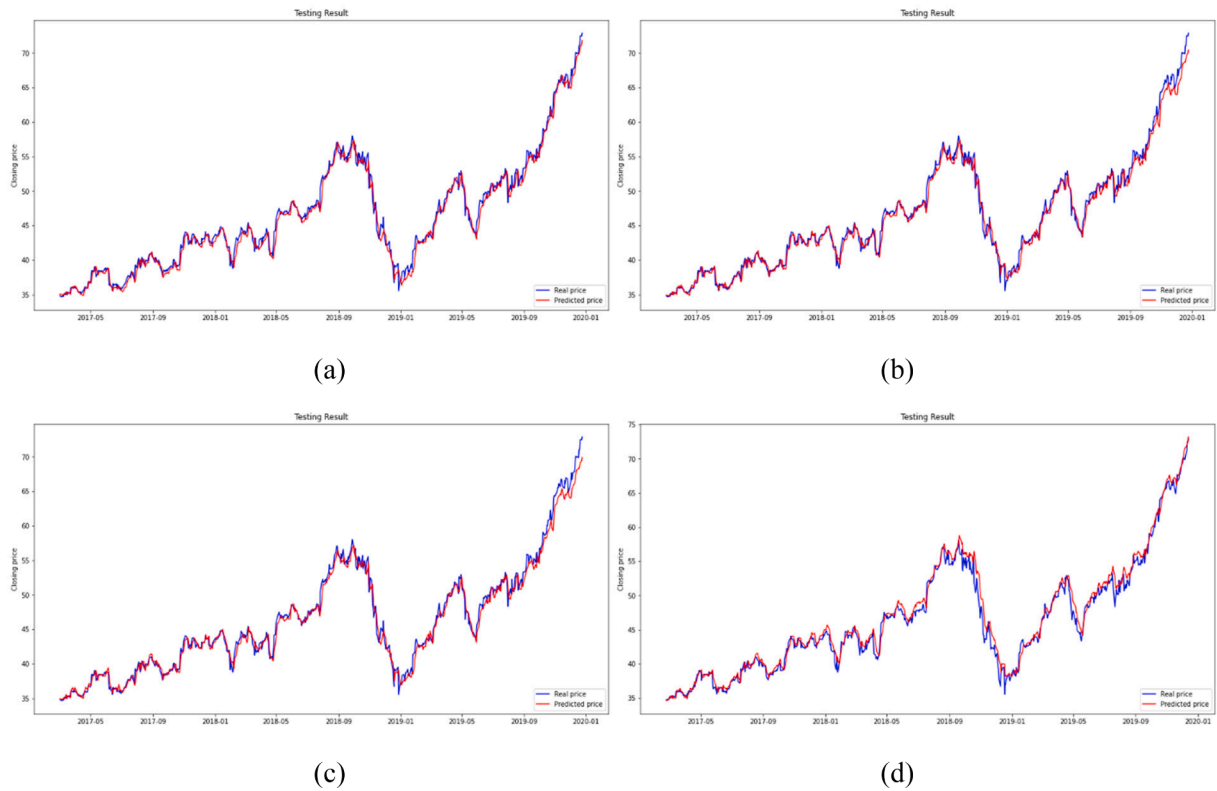


Fig. 5. Comparison of the predicted and real prices based on the test set (Apple stock). (a) The proposed method. (b) WGAN-GP. (c) Basic GANs. (d) LSTM.

Table 2

Results of k-fold cross-validation on test folds based on the proposed method (prediction 3 to 1).

Stocks	Mean			Variance		
	RMSE	MAE	R2-score	RMSE	MAE	R2-score
Apple	0.4840	0.3700	0.9540	0.0005	0.0001	0.0010
Nvidia	0.7334	0.6990	0.9734	0.0003	0.0018	0.0002
General Motors	0.6640	0.5060	0.9560	0.0011	0.0008	0.0004

5. Experiments

We evaluated the proposed framework using data from 12 stocks. This data was collected from Yahoo Finance and contained historical stock prices from 2010 to 2020. Following feature extraction, the input to the generator consisted of a set of 14 features, which were as follows:

- Historical data on the stock include High, Low, Open, Close, Adjacent close, and Volume.
- Some popular technical indicators were calculated based on stock price data. Among them, RSI (relative strength index), Stochastic RSI, EMA (exponential moving average), logarithmic momentum,

MACD (moving average convergence/divergence), and Bollinger Bands can be mentioned.

All of these features were normalized to map their values to the range of 1 and -1 . We considered 70 % of the data for training and 30 % for testing. Then, the windowing step was performed separately for training and testing data. For instance, a window with a length of 3 was slid over the data, obtaining segments with dimensions of 3×14 . These segments were fed to the generator as input, where 14 represents the number of features. The purpose of this experiment was to forecast the next day's price based on the information from three consecutive days. Therefore, the generator produced the price for the next day. In the conditioning step, the predicted price was concatenated with the price of three consecutive days and fed to the discriminator as fake input. Additionally, real input of the discriminator was provided by concatenating the target price with the price of three consecutive days. The target price is the true price of the next day and can be extracted from the training and test sets.

As previously mentioned, GANs are a min-max game between two models: the discriminator and the generator. This game may not converge to a global Nash equilibrium and only reach desirable local points. In practice, Nash equilibrium refers to finding local optimal points for the optimization functions in Eqs. (6) and (7). This concept can be equivalent to local optimal points in standard neural networks. The reasonable results of our method demonstrate that the model

Table 3

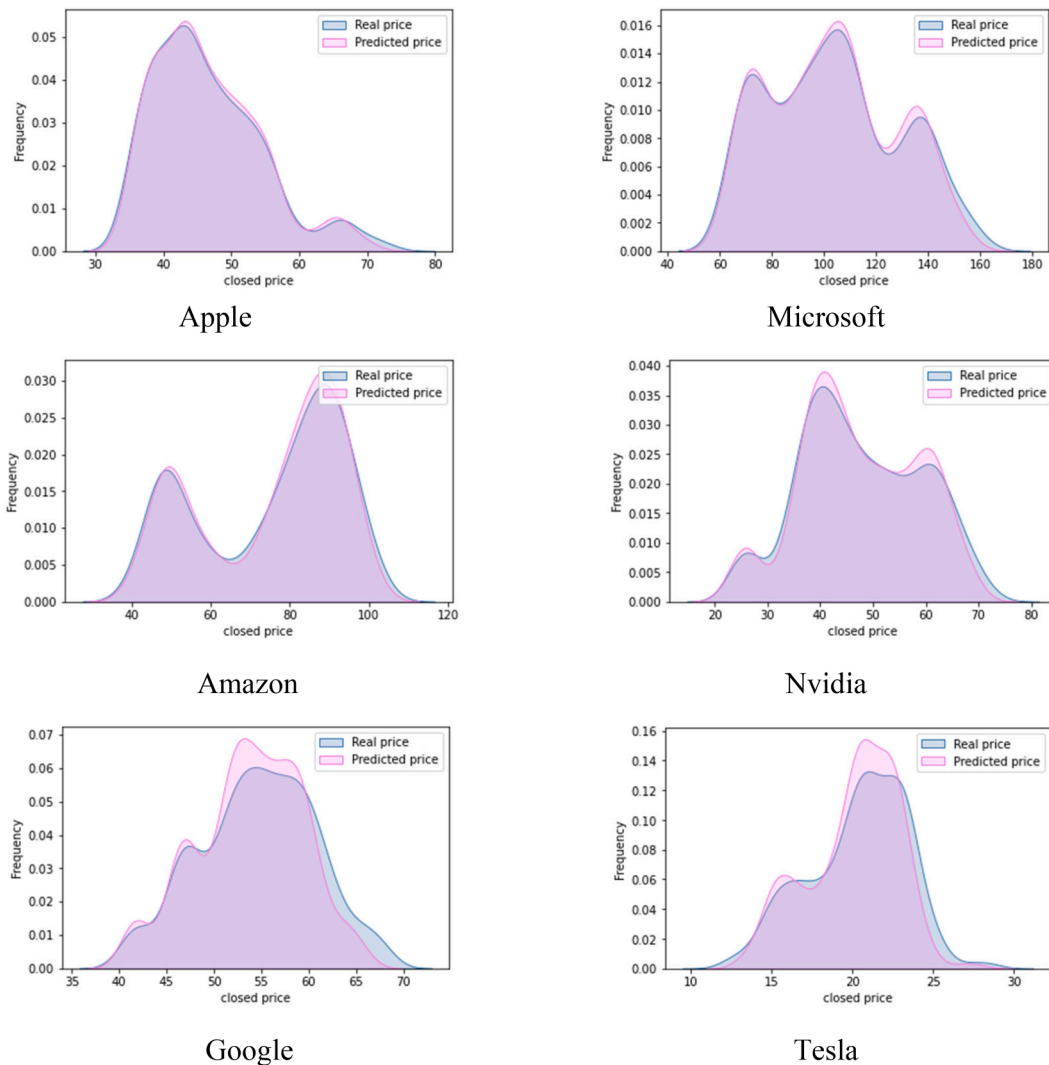
Comparison of the results for many-to-many mapping on the test set (on Apple stock).

Method	Training RMSE				Testing RMSE			
	10 to 1	10 to 2	10 to 3	10 to 5	10 to 1	10 to 2	10 to 3	10 to 5
Proposed method	0.54	0.55	0.54	0.65	1.01	1.19	1.51	1.53
WGAN-GP	0.47	0.48	0.59	0.73	1.04	1.23	1.66	1.73
Basic GANs	0.52	0.62	0.63	0.67	1.05	1.26	1.71	1.75
LSTM	0.58	0.43	0.58	0.52	1.03	1.23	1.54	1.54

Table 4

Summary of common GANs-based studies for stock price forecasting.

Model	Methodology	Features	Dataset	Results
Zhou et al. (2018)	Basic GANs with LSTM as generator and CNN as discriminator.	Historical prices + technical indicators. (13 features)	42 stocks in China stock market. Jan. 2016 - Dec. 2016. Minute time frame.	Different values based on metrics other than RMSE.
Zhang et al. (2019)	Basic GANs with LSTM as generator and MLP as discriminator.	Historical prices + MA5. (7 features)	S&P 500, IBM, Microsoft, ShanghaiComposite Index, Ping An Insurance Company of China (PAICC) 1998 – 2018. Daily time frame.	RMSE: 4.103 on Avg.
Sonkiya et al. (2021)	Basic GANs with GRU as generator and CNN as discriminator + sentiment analysis.	Historical prices + technical indicators + stock indexes of some countries + several commodities. (37 features)	Apple. July 2010 - July 2020. Daily time frame.	RMSE: 1.827
Lin et al. (2021)	WGAN-GP with GRU as generator and CNN as discriminator + sentiment analysis.	Historical prices + technical indicators + stock indexes of some countries + several commodities. (36 features)	Apple. July 2010 - July 2020. Daily time frame.	RMSE: 3.880
The proposed method	DRAGAN with GRU as generator and CNN as discriminator + feature matching.	Historical prices + technical indicators. (14 features)	12 stocks. July 2010 - July 2020. Daily time frame.	RMSE: 0.92 on Apple 1.105 on Avg.

**Fig. 6.** Predicted prices distribution versus real prices distribution (for the proposed method).

converges to desirable local optimal points. Fig. 4.a presents an example of the discriminator loss for Apple stock. The loss begins with high values during the initial training steps and fluctuates within a narrow range in the final iterations (at the end of the training, the values of

discriminator loss are not entirely constant and have very small fluctuations). Similarly, the RMSE error decreases during training and reaches a relatively stable point at the final iterations (Fig. 4.b).

The model was trained using Adam optimizer based on the proposed

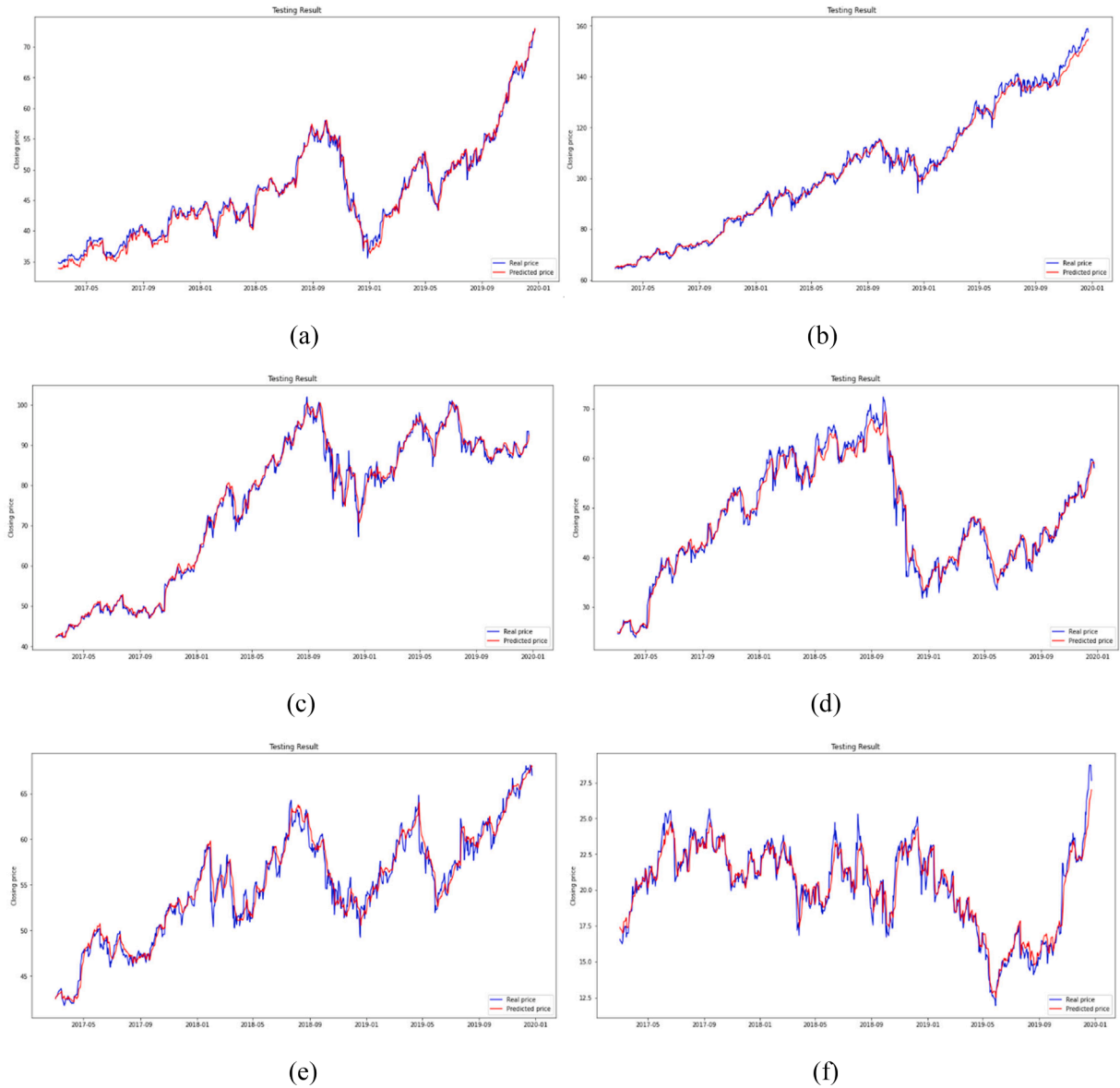


Fig. 7. The price prediction charts using the proposed method for various stocks. (a) Apple. (b) Microsoft. (c) Amazon. (d) Nvidia. (e) Google. (f) Tesla.

cost functions in Eqs. (6) and (7). The optimal values of the hyper-parameters, including $\lambda_1 \sim 10$, $\lambda_2 \sim 1$, $k = 1$, and $c \sim 10$, were selected through a trial and error. After training, the generator was employed exclusively as the final forecasting model. Three metrics, Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and R2-score, were utilized to evaluate the performance of the proposed method.

The model's stopping criterion was based on the RMSE and validation error. The training was stopped when the RMSE error's fluctuations remained within a small range for a certain number of epochs (e.g., $|RMSE_{t+1} - RMSE_t| < \epsilon$ should hold for 10 consecutive epochs). That means the model training was terminated when the RMSE error's fluctuations remained small and the validation error did not decrease. Using validation error was a practical approach to prevent model overfitting (15 % of the training data was used for the validation set).

The proposed model was compared with LSTM as a baseline, basic GANs, and WGAN-GP. The LSTM model consisted of a bidirectional LSTM layer with 128 units and a fully connected layer on top of it. To ensure a fair comparison, we employed windowing, conditioning, and the same architecture across all GANs-based models (as described in section 4.5). The cost functions on Eqs. (1) and (3) were used for training

basic GANs and WGAN-GP, respectively.

Table 1 summarizes the evaluation results for all methods. According to these results, the proposed method outperforms baseline approaches. WGAN-GP typically achieves second place in terms of performance, although basic GANs and LSTM sometimes produce slightly better results. Fig. 5 depicts the predicted prices on the test set as stock charts, and these results are consistent with the error values. These charts demonstrate that the proposed method can accurately capture stock trends.

The issue of overfitting in GANs differs from standard models, and it is somewhat associated with mode collapse. The proposed approach reduced mode collapse using customized DRAGAN functions, conditional GANs, and feature matching. Therefore, the likelihood of overfitting had also been reduced. Furthermore, we employed common techniques such as dropout, weight regularization, normalization, and adding noise to the inputs in the discriminator and generator networks, significantly reducing overfitting. Nevertheless, k-fold cross-validation was employed on training data to assess the generalization of the proposed model and ensure it did not overfit. In this experiment, the value of k was set to 5, and the results were reported on test folds in Table 2

(due to its high computational cost, this evaluation was performed for only three stocks). The results demonstrate that by changing the training set, the error variance on the testing folds is small, indicating that the model exhibits good generalization ability.

Another experiment evaluated the proposed method's ability for many-to-many mapping. In this examination, we set the input window size to the generator as 10, intending to predict the upcoming prices based on the data of 10 consecutive days. This experiment predicted the prices of 1, 2, 3, and 5 days ahead. As shown in Table 3, the proposed method can learn many-to-many mapping better than the compared methods. Due to the strong correlation among all three evaluation metrics (Table 1) and to facilitate easier comparison, the table's results have only been presented based on RMSE. These results show that LSTM is ranked second in performance. In addition, comparing the results of Tables 1 and 3 indicates that LSTM can learn long-term dependencies better than the basic GANs and WGAN-GP methods.

A comprehensive examination of the results indicates that the proposed approach has effectively addressed the challenges of the problem. However, due to the training instability of GANs and the highly volatile nature of certain stocks, it is impossible to definitively determine the superiority among the three methods: basic GANs, WGAN-GP, and LSTM. Nonetheless, WGAN-GP generally ranks second. In the context of many-to-many mapping, LSTM outperforms both WGAN-GP and basic GANs. Additionally, it is worth noting that some instances of superiority have been solely based on the training set, which is undesirable as the method has displayed poor performance during the testing phase. Furthermore, it should be acknowledged that relying solely on conventional GAN designs does not guarantee reliable performance. Therefore, various aspects must be considered when utilizing GANs for stock prediction problems.

Additionally, our goal was to compare the results of the proposed method with those of previous studies. Although this comparison could not be fair due to the unavailability of their code details, the use of different datasets, and different time frames. Instead, we summarized the results of popular GANs-based studies by methodology, type of features, dataset, and reported experimental results in Table 4. WGAN-GP had outperformed other GANs variants in these studies, but the proposed method produced better results than WGAN-GP. Sonkiya and Lin conducted a study on Apple stock during the same period, and comparing their results indicated that the suggested method was promising. In addition, our method can be improved by incorporating sentiment analysis and more efficient features.

Furthermore, we investigated the ability of the method to learn real data distribution. Explorations revealed that the distributions of predicted and real prices were very close, indicating that the proposed method performed well in learning the real data distribution. In Fig. 6, an example of this comparison is presented. Finally, the price prediction charts for six stocks are illustrated in Fig. 7.

6. Conclusion

This paper proposes a novel framework for stock price forecasting based on DRAGAN and feature matching, which improves training stability and mitigates mode collapse. Windowing was employed to acquire temporal correlations by the generator. Also, conditional GANs were used to capture temporal correlations and correlations between prices and features. In addition, customized cost functions were suggested for training the generator and discriminator. Experiments on data from several stocks demonstrated that the proposed method outperformed other baseline methods such as LSTM, basic GANs, and WGAN-GP.

To our knowledge, GANs-based studies in this area mainly utilize basic GANs and WGAN variants. Training instability and non-convergence are the main barriers to developing GANs for stock price forecasting. Therefore, promising results of the suggested approach can motivate future research. It is worth mentioning that a limited number of studies have employed sentiment analysis of news in combination

with GANs models, but their results need further examination due to insufficient data (Lin et al., 2021; Sonkiya et al., 2021). Thus, one suggestion to improve the model is to investigate the use of sentiment analysis and more appropriate features. Moreover, we must explore GANs-based methods on more extensive and diverse datasets.

7. Code availability

The source code of our paper required to reproduce the predictions and results is available in the public GitHub repository.⁶

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Areekul, P., Senjyu, T., Toyama, H., & Yona, A. (2009). A hybrid ARIMA and neural network model for short-term price forecasting in deregulated market. *IEEE Transactions on Power Systems*, 25(1), 524–530.
- Arjovsky, M., & Bottou, L. (2017). Towards principled methods for training generative adversarial networks. *arXiv Preprint arXiv:1701.04862*.
- Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein generative adversarial networks. *Proceedings of the 34th International Conference on Machine Learning*, 214–223.
- Basak, S., Kar, S., Saha, S., Khaidem, L., & Dey, S. R. (2019). Predicting the direction of stock market prices using tree-based classifiers. *The North American Journal of Economics and Finance*, 47, 552–567.
- Bhuriya, D., Kaushal, G., Sharma, A., & Singh, U. (2017). Stock market prediction using a linear regression. *International Conference of Electronics, Communication and Aerospace Technology (ICECA)*, 2, 510–513.
- Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time series analysis: Forecasting and control*. John Wiley & Sons.
- Bustos, O., & Pomares-Quimbaya, A. (2020). Stock market movement forecast: A systematic review. *Expert Systems with Applications*, 156, Article 113464. <https://doi.org/10.1016/j.eswa.2020.113464>
- Cao, H., Lin, T., Li, Y., & Zhang, H. (2019). Stock Price Pattern Prediction Based on Complex Network and Machine Learning. *Complexity*, 2019, 1–12.
- Cao, J., & Wang, J. (2019). Stock price forecasting model based on modified convolution neural network and financial time series analysis. *International Journal of Communication Systems*, 32(12), 1–13.
- Chen, S., & Ge, L. (2019). Exploring the attention mechanism in LSTM-based Hong Kong stock price movement prediction. *Quantitative Finance*, 19(9), 1507–1515. <https://doi.org/10.1080/14697688.2019.1622287>
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv Preprint arXiv:1406.1078*.
- Chong, E., Han, C., & Park, F. C. (2017). Deep learning networks for stock market analysis and prediction: Methodology, data representations, and case studies. *Expert Systems with Applications*, 83, 187–205.
- Di Persio, L., & Honchar, O. (2017). Recurrent neural networks approach to the financial forecast of Google assets. *International Journal of Mathematics and Computers in Simulation*, 11, 7–13.
- Ebadati, O. M. E., & Mortazavi, M. T. (2018). An efficient hybrid machine learning method for time series stock market forecasting. *Neural Network World*, 28(1), 41–55.
- Fenghua, W. E. N., Jihong, X., Zhifang, H. E., & Xu, G. (2014). Stock price prediction based on SSA and SVM. *Procedia Computer Science*, 31, 625–631.
- Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 2672–2680.
- Gu, Q., & Dai, Q. (2021). A novel active multi-source transfer learning algorithm for time series forecasting. *Applied Intelligence*, 51(3), 1326–1350. <https://doi.org/10.1007/s10489-020-01871-5>
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. C. (2017). Improved training of Wasserstein GANs. *Advances in Neural Information Processing Systems*, 30, 5769–5779.

⁶ <https://github.com/fashahabi/DRAGAN-Feature-Matching>.

- Gunduz, H. (2021). An efficient stock market prediction model using hybrid feature reduction method based on variational auto encoders and recursive feature elimination. *Financial Innovation*, 7(1), 1–24. <https://doi.org/10.1186/s40854-021-00243-3>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Jin, Z., Yang, Y., & Liu, Y. (2020). Stock closing price prediction based on sentiment analysis and LSTM. *Neural Computing and Applications*, 32(13), 9713–9729. <https://doi.org/10.1007/s00521-019-04504-2>
- Kodali, N., Abernethy, J., Hays, J., & Kira, Z. (2017). On convergence and stability of GANs. *arXiv Preprint arXiv:1705.07215*.
- Kumbure, M. M., Lohrmann, C., Luukka, P., & Porras, J. (2022). Machine learning techniques and data for stock market forecasting: A literature review. *Expert Systems with Applications*, 197, Article 116659.
- Lin, H. C., Chen, C., Huang, G. F., & Jafari, A. (2021). Stock price prediction using generative adversarial networks. *Journal of Computer Science*, 17, 188–196. <https://doi.org/10.3844/jcssp.2021.188.196>
- Mirza, M., & Osindero, S. (2014). Conditional generative adversarial nets. *arXiv Preprint arXiv:1411.1784*.
- Nasiri, H., & Ebadzadeh, M. M. (2022). MFRFNN: Multi-Functional Recurrent Fuzzy Neural Network for Chaotic Time Series Prediction. *Neurocomputing*, 507, 292–310.
- Nguyen, T.-T., & Yoon, S. (2019). A novel approach to short-term stock price movement prediction using transfer learning. *Applied Sciences*, 9(22), 4745. <https://doi.org/10.3390/app9224745>
- Pai, P. F., & Lin, C. S. (2005). A hybrid ARIMA and support vector machines model in stock price forecasting. *Omega*, 33(6), 497–505.
- Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert Systems with Applications*, 42(1), 259–268.
- Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., & Chen, X. (2016). Improved techniques for training GANs. *Proceedings of the 30th International Conference on Neural Information Processing Systems*, 2234–2242.
- Sedighi, M., Jahangirnia, H., Gharakhani, M., & Farahani Fard, S. (2019). A novel hybrid model for stock price forecasting based on metaheuristics and support vector machine. *Data*, 4(2), 75.
- Sinaga, F. M., Jonas, M., & Halim, A. (2019). Stock trend prediction using SV-kNNC and SOM. *Fourth International Conference on Informatics and Computing (ICIC)*, 1–5. <https://doi.org/10.1109/ICIC47613.2019.8985731>
- Singh, R., & Srivastava, S. (2017). Stock prediction using deep learning. *Multimedia Tools and Applications*, 76(18), 18569–18584.
- Sonkiya, P., Bajpai, V., & Bansal, A. (2021). Stock price prediction using BERT and GAN. *arXiv Preprint arXiv:2107.09055*.
- Villani, C. (2009). *Optimal transport: Old and new*. Springer.
- Wang, H.-Y., Li, H., & Shen, J.-Y. (2019). A novel hybrid fractal interpolation-SVM model for forecasting stock price indexes. *Fractals*, 27(04), 1950055. <https://doi.org/10.1142/S0218348X19500555>
- Xu, Y., Chhim, L., Zheng, B., & Nojima, Y. (2020). Stacked deep learning structure with bidirectional long-short term memory for stock market prediction. *International Conference on Neural Computing for Advanced Applications*, 447–460. https://doi.org/10.1007/978-981-15-7670-6_37
- Zhang, W., Zhang, S., Zhang, S., Yu, D., & Huang, N. (2019). A novel method based on FTS with both GA-FCM and multifactor BPNN for stock forecasting. *Soft Computing*, 23(16), 6979–6994.
- Zhang, K., Zhong, G., Dong, J., Wang, S., & Wang, Y. (2019). Stock market prediction based on generative adversarial network. *Procedia Computer Science*, 147, 400–406. <https://doi.org/10.1016/j.procs.2019.01.256>
- Zhou, X., Pan, Z., Hu, G., Tang, S., & Zhao, C. (2018). Stock Market Prediction on High-Frequency Data Using Generative Adversarial Nets. *Mathematical Problems in Engineering*, 2018, 1–11.